



Savitribai Phule Pune University
Centre For Distance and Online Education

SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE
CENTRE FOR DISTANCE AND ONLINE EDUCATION

Program	Master of Computer Application	
Course	MJ Programming from First Principles	
Topic Name	A means to understand the built-in types in a uniform framework (practice session) part 2.1	
Topic Code	-	
Unit/Module No. & Name	10	User defined types
Self-Learning Hours	-	

Topic Details

S.No	Topic	Pg.No
1.0	Introduction	4
2.0	Meaning And Definition	5-7
3.0	Nature Or Type	8-10
4.0	Examples And Case Studies	11-13
5.0	Keywords And Touch Vocabulary	14



OBJECTIVE

1. This unit aims to help you understand how user-defined types allow a programmer to model real-world ideas inside a computer program.
2. Instead of using only loose numbers and strings, you will learn to group related values into named types that match real things like students, songs, playlists, patients, and appointments.
3. When user-defined types are used properly, the code becomes easier to read, safer to run, and more reliable.
4. User-defined types also let you extend the programming language without changing the language itself.
5. You do not modify the compiler or create new syntax; you simply use features like structs, classes, and enums to build your own models.
6. In this way, all the types in your program become a small “language” you design for your own problem.
7. After completing this unit, you should be able to explain, in clear and simple sentences, what a data type and a user-defined type are.
8. You should be able to read a short problem description, such as a music app or hospital system, and identify the main nouns that can become types.
9. For each noun, you should be able to suggest important fields like name, id, or durationInSeconds.
10. You should be able to explain the difference between built-in types and user-defined types and give at least two reasons why both are important.
11. You should know the main kinds of user-defined types, such as records, classes, and enumerations.
12. You should also be able to describe, in simple words, how a small MP3 music library can be designed using these user-defined type ideas.

1.0 INTRODUCTION

1.1 Why We Study Data Types

Every computer program works with some kind of data. A calculator uses numbers. A chat app uses messages, usernames, and time stamps. A music player uses songs, artists, and playlists. When you write a program, you must tell the computer what kind of data you want to store and what you want to do with that data. This is what a data type does.

Most programming languages give you basic types like integer, float, character, string, and Boolean. These are like small building blocks. They are useful but not enough to represent real-life things. For example, a college student is not only a name — a student has an ID, a list of courses, grades, and contact details.

User-defined types let you create your own bigger blocks. You can group related pieces of data together and give that group a clear name. The computer then treats it as a new type, similar to int or string. This means you design your own types to match the problem you are solving.

As programming tasks grow, handling many separate variables becomes difficult. You may find many variables always belonging together in pairs or groups. At this point, user-defined types become very useful. They give structure to your program and make your code easier to understand.

1.1.1 Real World And Programs

There is a strong link between the real world and the world inside a computer program. When we talk about modeling, we mean creating a simple version of real things so that a computer can work with them. A map is a small model of a city. In the same way, a user-defined type is a model of some real or imaginary thing inside a program.

You cannot store a real person inside a computer, but you can store important information about the person. You choose which details to include and how to arrange them. If you choose wisely, your type becomes simple for the computer but also useful for your program. Good modeling is about choosing what to include and what to leave out.

For example, with an MP3 track, the actual sound is stored in a file. But the program also needs details like the song title, artist name, album name, track length, and file location. A user-defined type called Track lets you store all these details neatly in one place.

2.0 MEANING AND DEFINITION

2.1 Meaning Of Data Type

In computer science, a data type is a rule that tells us what kind of value we have and what operations we can perform on that value. An integer type allows addition and subtraction. A string type allows operations like joining two strings or searching for a letter. The type helps the compiler or interpreter check that you are using values in a safe and meaningful way.

We can think of a type as a label plus a set of allowed actions. When we declare a variable with a type, we are saying, “This box will hold values of this shape, and I will use them in these ways.” If we try to use the variable in a wrong way, the type system can warn us. In this sense, the type system works like an assistant that looks over your code and points out likely mistakes.

2.1.1 Meaning Of User Defined Type

A user defined type is a data type that is created by the programmer, using language features such as structures, classes, records, or enumerations. While primitive types are supplied by the language designers, user defined types come from the needs of a specific problem. Whenever the simple types are not enough to express an idea clearly, the programmer can invent a new type that fits that idea.

We define a user defined type by giving it a name and describing its internal structure. The structure can be made from primitive types and from other user defined types. Once we have defined such a type, we can declare variables of that type, pass them to functions, return them from methods, and store them in collections. In this way, the new type becomes a full citizen of the language, almost as if the language had always had it.

In an MP3 player, a Track type might have fields like title, artist, album, durationInSeconds, and filePath. Taken together, these fields form one value that stands for “one MP3 track” inside the program.

2.2 Formal View Of User Defined Types

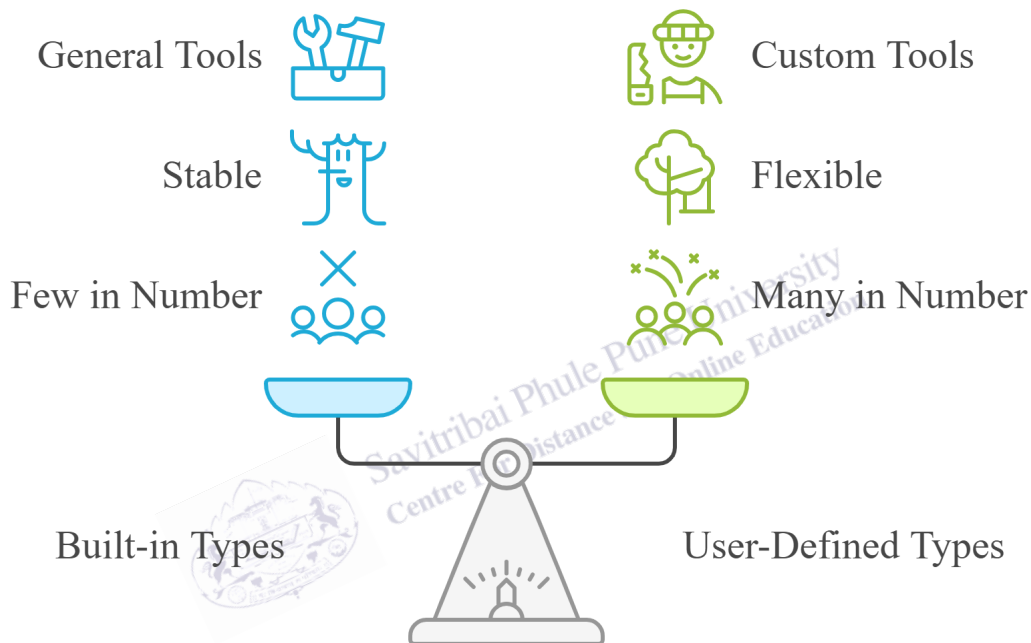
In a more formal view, a user defined type describes a set of possible values and a set of operations that make sense for those values. The values are built from other types using builders such as “record of,” “list of,” or “either this or that.” The operations may include reading and writing fields, comparing two values, or applying domain specific rules, such as checking that a track duration is positive.

2.2.1 Built In And User Defined Types

Built in types are general tools that work in almost every program. They are few in number, stable, and highly tuned by the language implementers. User defined types are custom tools that match one family of problems well. They can be many, flexible, and ready to change as the design of the system changes.

Fig. 1

Balancing Built-in and User-Defined Types



This diagram shows how built-in and user-defined data types balance each other. Built-in types are stable, few, and general-purpose, while user-defined types are flexible, custom-made, and numerous, helping programmers design solutions tailored to specific problems.

In practice, we combine both. Primitive types handle low level details such as counts, indices, and simple flags. User defined types handle higher level ideas such as Customer, Order, Playlist, or Appointment. In our MP3 model, integers are still used for things like duration in seconds and index positions, but the main story is told in terms of Track, Artist, Album, and Playlist types.

2.2.2 Role Of The Type System

A type system is the set of rules that a language uses to connect types with values and expressions. Some type systems are static, meaning they check most rules before the program runs. Others are dynamic, meaning they check many rules while the program is running. Many modern languages mix both ideas so that they can catch many errors early and still remain flexible.

For user defined types, the type system helps catch mismatches. If a function expects a Track and you pass a Patient, the compiler can stop you. If a field is declared as an enumeration and you try to store a number outside the allowed set, the system can complain. In this way, the type system supports the programmer and gives extra safety, especially as the program grows large.



3.0 NATURE OR TYPE

3.1 Main Kinds Of User Defined Types

User defined types appear in many forms, depending on the language and the style of programming. The most common forms are records or structs, classes and objects, enumerations, and variants or unions. Each form has its own nature and is suited to certain modeling tasks. Learning when to use each kind is part of learning to design models well.

3.1.1 Records Or Structs

A record, often called a struct in languages like C, is a simple user defined type that groups several named fields. Each field has its own type. All fields are stored together in memory, often in a fixed layout. Records are good for modeling simple objects that mostly store data and do not need complex behavior.

For example, a record for a student might contain an integer field for the student id, a string field for the name, and another string field for the major. Functions that work on students can take a record as a parameter, but the record itself does not carry methods. This keeps the data and behavior separate and easy to see. A Track type in a low level language might also be written as a struct that holds fields for title, artist, and duration.

3.1.2 Classes And Objects

A class is a more powerful user defined type that supports both data and methods in one unit. An object is a concrete instance of a class. Classes are central to object oriented programming languages. They allow ideas like encapsulation, which means hiding inner details, and inheritance, which means sharing behavior between related classes.

When we say that user defined types are a means to model real systems, classes are often what we imagine. A class can hide its internal fields and show only a clean interface. This gives other parts of the program a simple way to use the model without needing to know how it is built inside. A Track class in an object oriented MP3 player might include methods such as play, pause, displayDetails, and addToPlaylist.

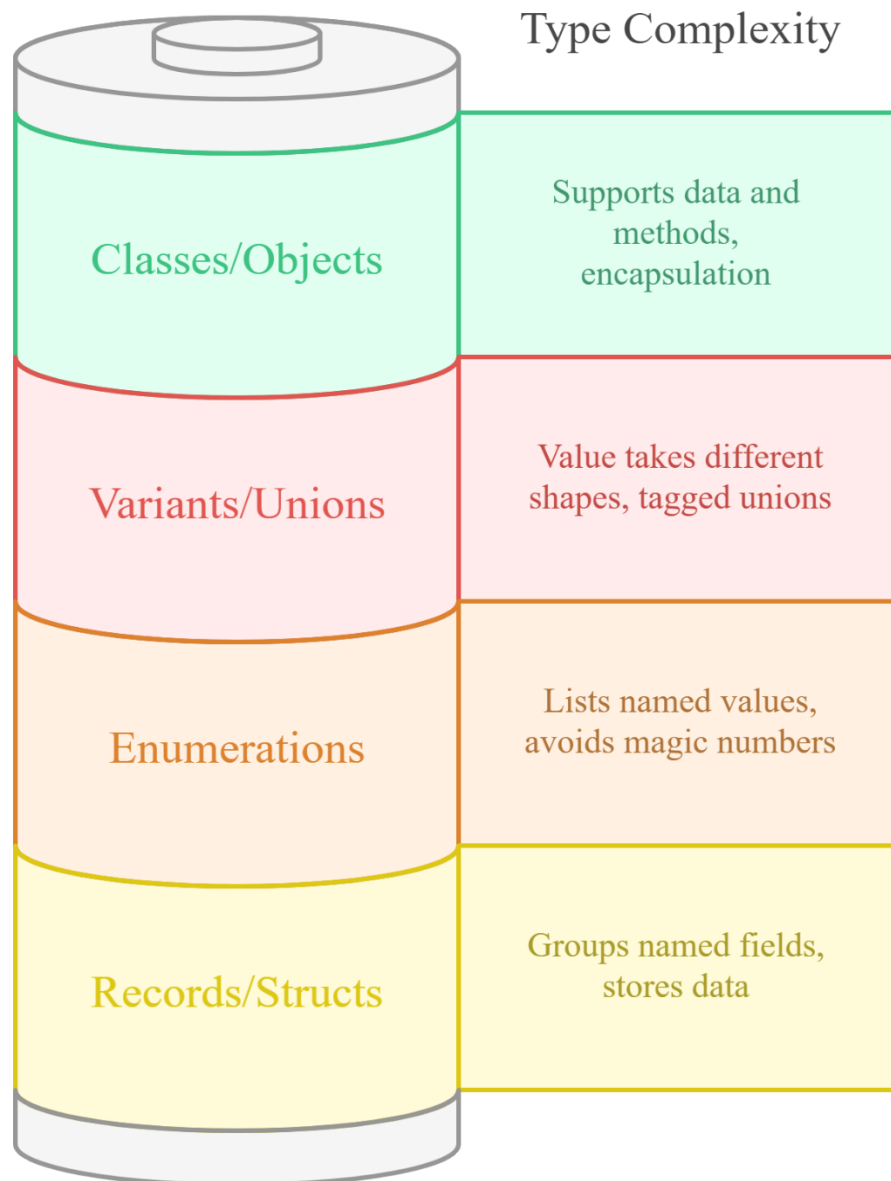
3.1.3 Enumerations

An enumeration, or enum, is a user defined type that lists a small set of named values. It is useful when a variable can only take one value from a limited list, such as days of the week, levels of access, or states of a process. Enums make code safer and clearer because they avoid magic numbers and repeated string names that can be typed wrongly.

For example, instead of using 0, 1, and 2 to mean Stopped, Playing, and Paused, a music player program can define an enum called PlayerState with members STOPPED, PLAYING, and PAUSED. The compiler can then check that only these states are used in the code. Reading such code feels like reading short English phrases, which helps both correctness and understanding.

Fig. 2

User defined types range from simple data to complex behavior.



This diagram explains how user-defined data types increase in complexity. It starts with simple records for storing data, moves to enumerations and variants for structured choices, and ends with classes that combine data and methods to support complex behavior.

3.1.4 Variants And Collections

A variant or union type allows a value to take one of several different shapes. At any time, the value has one active form, and the program must know which form is in use. Modern languages often provide safe variant types, sometimes called tagged unions, which carry both the data and a tag that says which case is active at the moment.

Variants are helpful when modeling data that can appear in several related forms. A media app may have a `MediaItem` type that can be a `Track`, a `Video`, or a `PodcastEpisode`. Each case may use different fields, but all share some common behavior, such as “play” and “showDetails.” Collections, such as lists or arrays of these items, then let the program manage many pieces of data at once.

3.2 Mutable And Immutable Types

Another part of the nature of a type is whether its values can change after creation. A mutable type allows its fields to be updated. An immutable type does not allow such changes; instead, any update creates a new value. Immutable types are common in functional programming and can reduce bugs related to unexpected changes in shared data.

For user defined types, you should decide which fields, if any, may change. For example, a `Patient` type might allow the address to change over time, but it should not allow the id or date of birth to change. In an MP3 library, a `Track` type might be immutable, because the title and duration of a song rarely change. A `Playlist`, on the other hand, may be mutable, because users often add or remove tracks.

4.0 EXAMPLES AND CASE STUDIES

4.1 Simple Examples

To see how user defined types help us model real situations, it is useful to start with a few simple examples. Each example shows how a type brings scattered pieces of data together into a single, named unit.

4.1.1 Example Of A Student Type

Suppose we want to build a small system to track students in a course. If we try to use only primitive types, we might have separate arrays for student ids, names, and grades. This is hard to manage and easy to break, because a mistake in indexing can mix up one student with another.

Instead, we define a user defined type named Student with fields such as id, name, email, and grade. Now a single Student value keeps related data together. When we pass a Student to a function that prints a report or updates a grade, we know that all needed data travels as a unit. The Student type becomes a new word in the language of the program.

4.1.2 Example Of A Mp3 Track Type

Now consider an MP3 music player. We can define a Track type with fields such as title, artist, album, durationInSeconds, and filePath. Each Track value stands for one song in the library. The filePath tells the player where to find the audio data. The other fields are used to show the user information about the song and to sort or search the library.

Because all of these facts are collected into one Track type, the program can work at a higher level. Instead of passing separate strings and numbers, it can pass a single Track value to functions or methods. This makes the code easier to read and reduces the chance of mixing up fields.

4.1.3 Example Of A Playlist Type

A Playlist type can hold a name and an ordered list of tracks. The list may be a simple array or a more advanced collection. The Playlist may also have behavior, such as methods to addTrack, removeTrack, and moveTrackUp. Playlists are the way many users think about their music, so modeling them as user defined types is natural.

4.2 Case Study Of A Mp3 Music Library

4.2.1 Data Types In The Library

In a small MP3 music library system, we might have several core types: Track, Artist, Album, Playlist, and MediaLibrary. The MediaLibrary type could hold lists of all tracks, artists, and playlists. The Artist type could include the artist name and a list of tracks by that artist. The Album type might include the album title, year, and a list of tracks in track order.

Each of these types mirrors something in the user's mental picture of their music collection. Taken together, they form a rich model. By designing these types carefully, we make later tasks, such as searching for a song or creating a new playlist, much easier to implement.

4.2.2 Operations On The Library

Once the types are in place, many operations become simple. To find all tracks by a given artist, we can search the list of Artist values and then use the list of tracks stored in the matching artist. To play a playlist, we simply go through its list of Track values, one by one. To remove an album, we can remove its tracks from the main library, using the links from Album to Track.

The MP3 library case study shows clearly how user defined types are a means to model and organize complex data.

4.3 Case Study Of A Hospital System

4.3.1 Patient, Doctor, And Appointment Types

Our second case study is a small hospital information system. Here, the key actors are patients, doctors, and appointments. User defined types help us keep each of these ideas clear and separate while also linking them in the right way.

We can define a Patient type that stores id, name, dateOfBirth, bloodGroup, and address. A Doctor type may have id, name, specialty, and a list of assigned patients. An Appointment type can connect a patient, a doctor, a date, and a status field that uses an enumeration such as SCHEDULED, COMPLETED, or CANCELLED.

By tying these types together, we get a rich model of the system. When a new appointment is created, the program creates an Appointment value that links to one Patient and one Doctor. When we print a schedule, we loop over Appointment values rather than over separate lists of names and times. The program can now speak in terms of Patients, Doctors, and Appointments, just as the hospital staff do in real life.

4.4 Common Mistakes And Good Practices

4.4.1 Typical Errors In Modeling

Beginners often make a few common mistakes when designing user defined types. One mistake is to use only primitive types, even when a new type would make the code clearer. Another mistake is the opposite: creating too many tiny types that do not add meaning, such as wrapping an integer id in a separate type that is never given more structure or behavior.

A further mistake is to mix unrelated ideas in one type. For example, a Student type that also stores details about the last exam room and the cafeteria balance may be carrying too much. In such cases, it is better to create smaller types, such as ExamRecord and PaymentAccount, and link them to the Student type.

4.4.2 Guidelines For Better Designs

Good modeling tries to give each type one main purpose. Names should be clear, fields should match real facts, and relationships between types should reflect real relationships in the world. In a music system, Track, Artist, Album, and Playlist should each have their own clear roles. In a hospital system, Patient, Doctor, and Appointment should do the same.

When in doubt, ask yourself what nouns appear in the problem story. These nouns are good candidates for user defined types. Then ask what facts belong to each noun and what actions should be done with it. Over time, this simple habit will make your designs stronger and your code easier to understand.

These case studies show that one clear set of types can support many tasks. The same modeling ideas appear again in different domains, such as music and health care. When you see this pattern, it becomes easier to trust and reuse your designs over time better.

1. User defined types help programmers model real world ideas inside programs.
2. Primitive types are basic building blocks; user defined types build richer models on top of them.
3. Records, classes, enums, and variants are common forms of user defined types.
4. Good models keep each type focused on one main purpose and match real nouns in the problem.
5. An MP3 library can be modeled with Track, Artist, Album, Playlist, and MediaLibrary types.
6. A hospital system can be modeled with Patient, Doctor, and Appointment types.

5.0 Keyword Touch Vocabulary Meaning

Keyword	Simple Meaning In This Unit
Data type	Rule about what kind of value we have and how we can use it
Primitive type	Basic built in type such as integer, float, or string
User defined type	Type created by the programmer for a specific problem
Record / struct	Group of named fields stored together as a single value
Class	Type that combines data fields with methods
Object	One actual instance built from a class
Enumeration	Type with a small fixed set of named states
Variant / union	Type whose value can take one of several related shapes
Track	User defined type that models one MP3 song in the library
Playlist	User defined type that stores an ordered list of track values
Appointment	User defined type that links patient, doctor, time, and status